Modul Pelatnas IOAI Indonesia

Modul 4: Pengenalan Computer Vision

Tim Pembina IOAI Indonesia
sc.ioai.id@gmail.com

Juli 2025

Modul Pelatnas IOAI Indonesia

Modul 4: Pengenalan Computer Vision

Penyusun

Tim Pembina IOAI Indonesia

sc.ioai.id@gmail.com

Daftar Isi

Pengantar III

1 Pengenalan Pengolahan Citra Digital 1
- 1.1 Apa itu Citra Digital? 1
- 1.2 Membaca dan Menampilkan Citra 2
- 1.3 Operasi Dasar pada Citra 3
- 1.4 Transformasi Warna dan Histogram 4
- 1.5 Praproses Citra 6

2 Klasifikasi Citra (Image Classification) 9
- 2.1 Pengantar 9
- 2.2 Convolutional Neural Networks (CNN) 9
- 2.3 Arsitektur CNN Modern: ResNet dan Lainnya 13

3 Deteksi Objek 17
- 3.1 Pengantar Deteksi Objek 17
- 3.2 Metrik Evaluasi untuk Object Detection 17
  - 3.2.1 Intersection over Union (IoU) 17
  - 3.2.2 Precision dan Recall 18
  - 3.2.3 F1 Score 18
  - 3.2.4 Average Precision (AP) dan mAP 18
- 3.3 YOLO: You Only Look Once 19

4 Segmentasi Citra (Image Segmentation) 25
- 4.1 Arsitektur Segmentasi Citra Berbasis Deep Learning 25
  - 4.1.1 FCN (Fully Convolutional Network) 25
  - 4.1.2 U-Net 26

4.1.3 SegNet ... 26
4.1.4 DeepLab (v3, v3+) ... 26
4.2 Implementasi Semantic Segmentation: Contoh dengan U-Net ... 26
4.3 Implementasi U-Net dengan PyTorch ... 26
4.4 Implementasi U-Net dengan PyTorch ... 29
4.5 Instance Segmentation dan Mask R-CNN ... 31

5 Topik Lanjut dalam Computer Vision ... 33
5.1 Transfer Learning dan Model Pretrained ... 33
5.2 Fine-Tuning Model untuk Dataset Baru ... 33
5.3 Augmentasi Data ... 34
5.4 Visualisasi dan Interpretasi CNN ... 34
5.5 Arsitektur Modern dan Perkembangan Terbaru ... 35
5.6 Aplikasi Computer Vision di Dunia Nyata ... 35
5.7 Referensi dan Bahan Bacaan Lanjutan ... 36

Bibliografi ... 37

Analytic Index ... 39

Pengantar

Puji syukur kehadirat Tuhan YME atas berkat limpahan rahmat dan karnuia-Nya, Buku Modul Pelatnas IOAI ini telah berhasil kami selesaikan. Buku Modul ini kami susun sebagai salah satu referensi rangkaian pembinaan/pelatihan nasional bagi siswa peserta didik yang mengikuti Pelatnas dalam rangka membentuk tim yang akan mewakili Indonesia pada ajang International Olympiad in Artificial Intelligence (IOAI).

Terima kasih yang sebesar-besarnya kami sampaikan kepada para Pembina, Asisten Pembina, dan para Alumni ajang OSN bidang Informatika dan IOI (TOKI), serta semua pihak yang telah berkontribusi sehingga Buku Modul ini dapat terwujud. Kami menyadari masih banyak kekurangan dalam penulisan Buku Modul ini. Untuk itu kami mohon maaf dan kami sangat mengharapkan masukan untuk perbaikan dan penyempurnaan selanjutnya, sehingga keberadaan Buku Modul ini dapat memberikan manfaat yang sebesar-besarnya bagi semua pihak.

Semoga Buku Modul Pelatnas IOAI ini dapat digunakan sebaik-baiknya untuk menunjang kegiatan Pelatnas IOAI dan mampu membantu menghasilkan calon-calon talenta Indonesia di bidang AI yang mampu memberikan prestasi yang membanggakan di tingkat internasional.

BAB 1
Pengenalan Pengolahan Citra Digital

1.1 Apa itu Citra Digital?

Citra digital (digital image) adalah representasi visual dari objek nyata yang disimpan dalam bentuk matriks nilai-nilai numerik. Setiap nilai tersebut disebut *piksel* (picture element), dan merepresentasikan intensitas cahaya atau warna di suatu titik koordinat pada citra.

Citra berwarna biasanya disimpan dalam format RGB, di mana setiap piksel memiliki tiga komponen: **R** (Red), **G** (Green), dan **B** (Blue), masing-masing dengan nilai antara 0 hingga 255. Sebaliknya, citra hitam-putih atau *grayscale* hanya memiliki satu channel intensitas.

Citra digital dapat disimpan dalam berbagai format, antara lain:

- **JPEG/JPG**: format kompresi lossy, umum digunakan untuk fotografi.
- **PNG**: format lossless, mendukung transparansi.
- **BMP**, **TIFF**: format tidak terkompresi, digunakan dalam aplikasi profesional.

Secara teknis, citra digital merupakan sebuah array 2D (untuk grayscale) atau 3D (untuk RGB), dengan dimensi:

- `(tinggi, lebar)` untuk grayscale.
- `(tinggi, lebar, 3)` untuk RGB.

Sebagai contoh, citra berukuran 128x128 piksel dengan 3 channel warna memiliki total $128 \times 128 \times 3 = 49.152$ nilai piksel.

—

Contoh Representasi Piksel:

```
Pixel RGB (45, 120, 200)
Artinya: Merah = 45, Hijau = 120, Biru = 200
```

Contoh representasi grayscale:

```
Pixel intensitas: 180
Artinya: Piksel agak terang (dalam skala 0-255)
```

1.2 Membaca dan Menampilkan Citra

Dalam pengolahan citra digital, langkah pertama yang umum dilakukan adalah membaca (load) citra dari file dan menampilkannya agar dapat divisualisasi dan dianalisis. Kita dapat menggunakan beberapa pustaka Python untuk tujuan ini, seperti:

- `OpenCV` – pustaka populer untuk Computer Vision yang efisien dan cepat.
- `Matplotlib` – pustaka visualisasi umum, juga dapat digunakan untuk menampilkan gambar.

Membaca Citra dengan OpenCV

Berikut contoh kode untuk membaca dan menampilkan citra:

Listing 1.1. Membaca dan menampilkan citra dengan OpenCV dan Matplotlib

```
import cv2
import matplotlib.pyplot as plt

# Membaca citra (dalam format BGR secara default oleh OpenCV)
img = cv2.imread('gambar.jpg')

# Konversi ke RGB untuk ditampilkan dengan matplotlib
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Menampilkan citra
plt.imshow(img_rgb)
plt.title('Contoh Citra')
plt.axis('off')
plt.show()
```

Catatan: OpenCV secara default membaca gambar dalam format BGR, bukan RGB, sehingga perlu dikonversi sebelum ditampilkan dengan `matplotlib`.

Membaca Citra dalam Grayscale

Jika ingin membaca citra dalam mode hitam-putih (grayscale), kita bisa menggunakan parameter tambahan:

Listing 1.2. Membaca citra grayscale

```
gray = cv2.imread('gambar.jpg', cv2.IMREAD_GRAYSCALE)

plt.imshow(gray, cmap='gray')
plt.title('Citra Grayscale')
plt.axis('off')
plt.show()
```

Dimensi dan Informasi Dasar Citra

Setelah membaca citra, kita bisa melihat bentuk dan tipe datanya:

```
print(img.shape)      # Output: (tinggi, lebar, 3)
print(gray.shape)     # Output: (tinggi, lebar)
print(img.dtype)      # Biasanya: uint8 (nilai 0-255)
```

Hal ini penting untuk memahami bagaimana data citra direpresentasikan dalam array numpy.

1.3 Operasi Dasar pada Citra

Operasi dasar pada citra digital meliputi transformasi bentuk, ukuran, dan orientasi citra. Transformasi ini penting dalam tahap awal pemrosesan atau praproses data visual sebelum diterapkan ke model machine learning atau deep learning.

1. Resize (Ubah Ukuran)

Mengubah ukuran citra sering dilakukan agar seluruh dataset memiliki dimensi seragam.

Listing 1.3. Resize citra ke ukuran tertentu

```
img_resized = cv2.resize(img, (100, 100))  # width x height

plt.imshow(cv2.cvtColor(img_resized, cv2.COLOR_BGR2RGB))
plt.title('Citra setelah Resize')
plt.axis('off')
plt.show()
```

2. Crop (Potong Citra)

Memotong bagian tertentu dari citra dilakukan dengan slicing array NumPy.

Listing 1.4. Crop citra (bagian tengah)

```
height, width = img.shape[:2]
crop_img = img[height//4:3*height//4, width//4:3*width//4]

plt.imshow(cv2.cvtColor(crop_img, cv2.COLOR_BGR2RGB))
plt.title('Citra setelah Crop')
plt.axis('off')
plt.show()
```

3. Rotate (Rotasi)

Rotasi dilakukan menggunakan transformasi affine. Kita bisa menentukan titik pusat dan sudut rotasi.

Listing 1.5. Rotasi 45 derajat

```
(h, w) = img.shape[:2]
center = (w // 2, h // 2)

# Tentukan matriks rotasi
M = cv2.getRotationMatrix2D(center, 45, 1.0)
rotated = cv2.warpAffine(img, M, (w, h))

plt.imshow(cv2.cvtColor(rotated, cv2.COLOR_BGR2RGB))
plt.title('Citra setelah Rotasi')
plt.axis('off')
plt.show()
```

4. Flip (Cermin)

OpenCV memungkinkan pembalikan citra secara horizontal atau vertikal.

- `flipCode = 0` → vertical
- `flipCode = 1` → horizontal
- `flipCode = -1` → horizontal + vertical

Listing 1.6. Flip horizontal

```
flipped = cv2.flip(img, 1)

plt.imshow(cv2.cvtColor(flipped, cv2.COLOR_BGR2RGB))
plt.title('Citra setelah Flip Horizontal')
plt.axis('off')
plt.show()
```

5. Flatten (Meratakan Citra ke Vektor)

Flatten digunakan untuk mengubah citra menjadi vektor 1D, misalnya sebagai input MLP.

Listing 1.7. Flatten citra RGB ke vektor 1D

```
flattened = img.flatten()
print(flattened.shape)  # Output: (tinggi X lebar X channel,)
```

—

Transformasi-transformasi dasar ini akan sangat sering digunakan dalam preprocessing dataset citra atau augmentasi data sebelum model dilatih.

1.4 Transformasi Warna dan Histogram

Dalam pengolahan citra, representasi warna dan intensitas sangat memengaruhi bagaimana informasi visual ditangkap oleh algoritma. Transformasi warna dan histogram sering

digunakan untuk meningkatkan kontras, menyederhanakan citra, atau menyesuaikan representasi ke dalam bentuk yang lebih berguna untuk klasifikasi.

1. Konversi Warna (RGB ke Grayscale)

Salah satu transformasi paling umum adalah konversi dari citra berwarna (RGB) ke citra grayscale (hitam putih). Grayscale menyederhanakan perhitungan dan mengurangi dimensi data, sangat berguna untuk deteksi tepi, segmentasi, dan preprocessing sederhana.

Listing 1.8. Konversi RGB ke Grayscale

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

plt.imshow(gray, cmap='gray')
plt.title('Citra Grayscale')
plt.axis('off')
plt.show()
```

Grayscale disusun berdasarkan intensitas, biasanya menggunakan rumus:

$$I = 0.2989 \cdot R + 0.5870 \cdot G + 0.1140 \cdot B$$

Rumus ini menyesuaikan dengan persepsi manusia terhadap terang-gelap warna.

2. Histogram Citra

Histogram citra merepresentasikan distribusi intensitas piksel. Ini adalah alat penting untuk analisis kontras dan pencahayaan citra.

Listing 1.9. Menampilkan histogram citra grayscale

```
plt.hist(gray.ravel(), bins=256, range=[0,256])
plt.title('Histogram Intensitas Citra Grayscale')
plt.xlabel('Intensitas Piksel')
plt.ylabel('Jumlah Piksel')
plt.show()
```

3. Histogram Equalization

Equalization adalah teknik peningkatan kontras dengan mendistribusikan ulang nilai intensitas agar merata. Ini berguna untuk citra dengan kontras rendah atau pencahayaan tidak merata.

Listing 1.10. Histogram Equalization

```
equalized = cv2.equalizeHist(gray)

# Bandingkan sebelum dan sesudah
plt.subplot(1,2,1)
plt.imshow(gray, cmap='gray')
plt.title('Original')
```

```
plt.subplot(1,2,2)
plt.imshow(equalized, cmap='gray')
plt.title('Equalized')
plt.show()
```

4. Konversi Warna Lain (HSV, LAB)

Dalam beberapa kasus, representasi warna seperti HSV (Hue, Saturation, Value) atau LAB (Lightness, A-B channels) lebih berguna untuk segmentasi warna atau augmentasi.

Listing 1.11. Konversi ke HSV dan LAB

```
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
lab = cv2.cvtColor(img, cv2.COLOR_BGR2LAB)
```

—

Transformasi warna dan histogram merupakan bagian penting dari tahap praproses dan eksplorasi awal citra. Kombinasi konversi warna dan histogram equalization sering digunakan untuk meningkatkan kualitas data sebelum diproses lebih lanjut.

1.5 Praproses Citra

Praproses citra merupakan tahap penting dalam pipeline computer vision. Tujuannya adalah menyederhanakan informasi visual, mengurangi noise, dan menyesuaikan format citra agar sesuai dengan kebutuhan algoritma yang akan digunakan. Berikut adalah beberapa teknik praproses yang umum:

1. Normalisasi Nilai Piksel

Citra digital biasanya memiliki nilai piksel antara 0 hingga 255. Untuk mempercepat konvergensi model dan mencegah eksplosif nilai dalam pembelajaran mesin, nilai-nilai ini biasanya dinormalisasi ke rentang [0,1] atau memiliki rata-rata nol.

Listing 1.12.]Normalisasi nilai piksel ke rentang [0,1]

```
img_normalized = img / 255.0
```

Untuk normalisasi z-score (mean 0 dan standar deviasi 1), kita bisa gunakan:

```
mean = img.mean()
std = img.std()
img_standardized = (img - mean) / std
```

2. Denoising (Penghilangan Noise)

Noise bisa berasal dari sensor atau kompresi citra. OpenCV menyediakan metode denoising yang cukup efektif.

Listing 1.13. Gaussian Blur untuk meredam noise

```
blurred = cv2.GaussianBlur(img, (5, 5), 0)
```

Alternatif lain adalah median blur yang lebih efektif untuk noise salt-and-pepper:

```
denoised = cv2.medianBlur(img, 5)
```

3. Thresholding

Thresholding mengubah citra grayscale menjadi citra biner berdasarkan ambang tertentu. Ini berguna untuk segmentasi sederhana atau deteksi tepi awal.

Listing 1.14. Thresholding sederhana

```
_, binary = cv2.threshold(gray, 128, 255, cv2.THRESH_BINARY)

plt.imshow(binary, cmap='gray')
plt.title('Citra Biner')
plt.axis('off')
plt.show()
```

4. Adaptive Thresholding

Digunakan ketika pencahayaan dalam citra tidak merata, sehingga ambang global tidak cukup baik.

Listing 1.15. Adaptive Thresholding

```
adaptive = cv2.adaptiveThreshold(
    gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY, 11, 2
)
```

5. Morphological Operations

Operasi morfologi digunakan untuk memperbaiki bentuk objek, mengisi lubang, atau menghilangkan noise kecil.

Listing 1.16. Operasi morfologi: dilasi dan erosi

```
kernel = np.ones((5,5), np.uint8)
dilated = cv2.dilate(binary, kernel, iterations=1)
eroded = cv2.erode(binary, kernel, iterations=1)
```

—

Tahapan praproses ini sangat krusial untuk memastikan citra yang digunakan dalam training model bebas dari gangguan dan berada dalam format optimal untuk pembelajaran.

Referensi dan Bahan Bacaan Lanjutan

Untuk mempelajari lebih lanjut tentang pengolahan citra digital dan dasar-dasar computer vision, berikut beberapa sumber yang disarankan:

1. **PyImageSearch – Computer Vision and OpenCV Tutorials**
   Situs ini menyediakan banyak tutorial praktis dan penjelasan konsep dasar hingga lanjutan.
   https://pyimagesearch.com/
2. **OpenCV Documentation**
   Dokumentasi resmi OpenCV mencakup fungsi-fungsi penting dalam pemrosesan citra.
   https://docs.opencv.org/
3. **Digital Image Processing (3rd Edition) oleh Rafael C. Gonzalez dan Richard E. Woods**
   Buku klasik yang menjelaskan konsep dasar secara teoritis. Cocok untuk pembaca yang ingin pemahaman lebih formal.
4. **Practical Python and OpenCV + Case Studies (by Adrian Rosebrock)**
   Buku pemula yang aplikatif untuk memahami pemrosesan citra menggunakan Python.
   https://www.pyimagesearch.com/practical-python-opencv/
5. **Kaggle: Python Image Processing Notebooks**
   Kaggle menyediakan banyak contoh notebook interaktif yang bisa dijalankan di browser tanpa instalasi.
   https://www.kaggle.com/search?q=image+processing+notebook

BAB 2

Klasifikasi Citra (Image Classification)

2.1 Pengantar

Klasifikasi citra merupakan salah satu tugas paling mendasar dan penting dalam bidang computer vision. Tujuan dari klasifikasi citra adalah untuk menentukan label atau kategori dari sebuah gambar berdasarkan isi visualnya.

Sebagai contoh, sebuah sistem klasifikasi citra bisa digunakan untuk:

- Mendeteksi apakah suatu gambar menunjukkan anjing atau kucing.
- Mengenali jenis huruf dalam dokumen hasil scan.
- Mengklasifikasikan penyakit dari hasil citra medis seperti X-ray atau MRI.

Pada dasarnya, klasifikasi citra melibatkan dua langkah utama:

1. **Ekstraksi fitur**: Mengubah data piksel mentah menjadi representasi fitur yang lebih bermakna.
2. **Klasifikasi**: Menentukan kelas berdasarkan fitur yang diekstrak.

Pendekatan tradisional (sebelum era deep learning) menggunakan teknik seperti ekstraksi tepi, histogram warna, HOG (Histogram of Oriented Gradients), dan lain-lain. Namun kini, sebagian besar sistem klasifikasi citra modern menggunakan **Convolutional Neural Networks (CNN)** yang dapat melakukan ekstraksi fitur dan klasifikasi secara end-to-end.

Kita akan mempelajari CNN pada subbab berikut, serta bagaimana mengimplementasikannya untuk tugas klasifikasi citra.

2.2 Convolutional Neural Networks (CNN)

Convolutional Neural Networks (CNN) adalah arsitektur deep learning yang dirancang khusus untuk menangani data grid-seperti gambar. CNN terbukti sangat efektif untuk tugas-tugas seperti klasifikasi, deteksi objek, dan segmentasi citra.

1. Arsitektur CNN secara umum

CNN biasanya terdiri atas urutan dari layer berikut:

- **Convolutional layer**: melakukan operasi konvolusi antara filter (kernel) dan input untuk mengekstraksi fitur lokal.
- **Activation layer**: menerapkan fungsi aktivasi non-linear, biasanya ReLU.
- **Pooling layer**: mereduksi dimensi spasial sambil mempertahankan informasi penting (biasanya max pooling).
- **Fully connected layer**: seperti MLP, menggabungkan semua fitur menjadi vektor untuk klasifikasi.

2. Operasi Konvolusi

Operasi konvolusi dilakukan antara input citra dan filter (biasanya ukuran 3x3 atau 5x5). Setiap filter akan mengekstrak fitur tertentu dari input (misalnya tepi vertikal, tepi horizontal).

Jika I adalah citra input dan K adalah kernel, maka operasi konvolusi adalah:

$$S(i,j) = (I * K)(i,j) = \sum_m \sum_n I(i+m, j+n) \cdot K(m,n)$$

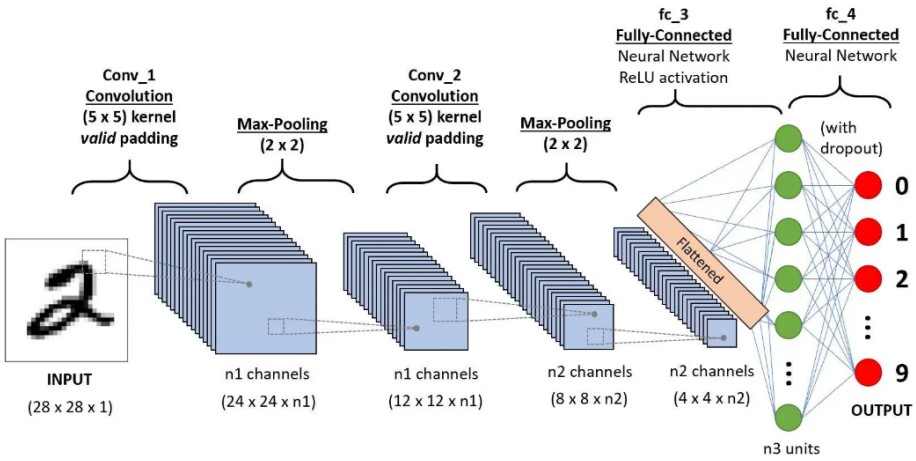
3. Pooling Layer

Pooling digunakan untuk mengurangi ukuran fitur map, memperkecil beban komputasi, dan membuat model lebih tahan terhadap pergeseran kecil pada citra.

Contoh paling umum adalah **Max Pooling**, yang mengambil nilai maksimum dari blok 2x2 atau 3x3.

4. Arsitektur CNN Sederhana

Berikut ini ilustrasi arsitektur CNN sederhana untuk klasifikasi citra 32x32:



Contoh alur:

Input (32x32x3) → Conv(3x3) → ReLU → MaxPooling → FC Layer → Softmax

6. Contoh Training dan Evaluasi CNN pada CIFAR-10

Berikut adalah implementasi lengkap mulai dari pemuatan data, pelatihan model, hingga evaluasi akurasi pada data uji:

Listing 2.1. Training dan evaluasi CNN sederhana pada CIFAR-10

```
import torch
import torchvision
import torchvision.transforms as transforms
import torch.optim as optim
import torch.nn as nn
import torch.nn.functional as F

# 1. Transformasi dan Load CIFAR-10
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])

trainset = torchvision.datasets.CIFAR10(root='./data',
            train=True, download=True, transform=transform)
trainloader = torch.utils.data.DataLoader(trainset,
    batch_size=64, shuffle=True, num_workers=2)

testset = torchvision.datasets.CIFAR10(root='./data',
    train=False, download=True, transform=transform)

testloader = torch.utils.data.DataLoader(testset,
    batch_size=100, shuffle=False, num_workers=2)

classes = ('plane', 'car', 'bird', 'cat', 'deer',
           'dog', 'frog', 'horse', 'ship', 'truck')

# 2. Arsitektur CNN Sederhana
class SimpleCNN(nn.Module):
    def __init__(self):
        super(SimpleCNN, self).__init__()
        self.conv1 = nn.Conv2d(3, 16, 3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(16, 32, 3, padding=1)
        self.fc1 = nn.Linear(32 * 8 * 8, 64)
        self.fc2 = nn.Linear(64, 10)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))  # 16x16x16
        x = self.pool(F.relu(self.conv2(x)))  # 32x8x8
```

```
        x = x.view(-1, 32 * 8 * 8)
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return x

# 3. Inisialisasi Model, Loss dan Optimizer
net = SimpleCNN()
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
net.to(device)

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(net.parameters(), lr=0.001)

# 4. Training Loop
for epoch in range(5):  # 5 epoch
    running_loss = 0.0
    for i, data in enumerate(trainloader, 0):
        inputs, labels = data[0].to(device), data[1].to(device)

        optimizer.zero_grad()
        outputs = net(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()
        if i % 100 == 99:
            print(f"[{epoch+1}, {i+1}] loss: {running_loss/100:.3f}")
            running_loss = 0.0

# 5. Evaluasi Akurasi
correct = 0
total = 0
with torch.no_grad():
    for data in testloader:
        images, labels = data[0].to(device), data[1].to(device)
        outputs = net(images)
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

print(f"Akurasi pada data uji: {100 * correct / total:.2f}%")
```

—

CNN telah merevolusi computer vision karena kemampuannya dalam mengenali pola visual dari citra mentah. Di bagian berikutnya, kita akan membahas arsitektur CNN modern seperti ResNet yang mengatasi beberapa kelemahan CNN tradisional.

2.3 Arsitektur CNN Modern: ResNet dan Lainnya

Seiring dengan semakin dalamnya arsitektur CNN (jumlah layer yang semakin banyak), muncul tantangan dalam proses pelatihan, seperti:

- **Degradasi performa**: menambah layer justru bisa memperburuk akurasi pada data uji.
- **Vanishing gradient**: gradient yang sangat kecil sehingga pembaruan bobot hampir tidak terjadi.

Untuk mengatasi hal ini, beberapa arsitektur CNN modern dikembangkan. Salah satu yang paling berpengaruh adalah **ResNet (Residual Network)**.

1. Motivasi Residual Network

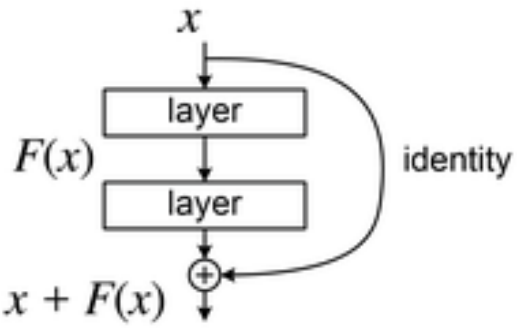
ResNet diperkenalkan oleh He et al. (2015) dan menjadi pemenang ImageNet 2015. Ide utamanya adalah mempermudah pelatihan jaringan sangat dalam (hingga 152 layer!) dengan memperkenalkan **shortcut connection** atau *residual connection*.

2. Blok Residual

Sebuah residual block memungkinkan jaringan untuk mempelajari "selisih" antara input dan output, bukan output-nya secara langsung.

Jika x adalah input dan $F(x)$ adalah transformasi (2 layer conv + ReLU misalnya), maka output blok residual adalah:

$$y = F(x) + x$$



Gambar 2.1. Blok residual sederhana (sumber: He et al., 2015)

Shortcut connection melewati layer nonlinear, dan gradient dapat mengalir dengan lebih mudah ke layer awal.

3. Arsitektur ResNet Populer

- **ResNet-18**: 18 layer, ringan dan cocok untuk banyak tugas kecil.

- **ResNet-50**: arsitektur standar dengan blok residual bottleneck.
- **ResNet-101, ResNet-152**: lebih dalam dan lebih kuat, tapi memerlukan lebih banyak komputasi.

4. Implementasi ResNet di PyTorch

PyTorch menyediakan implementasi bawaan dari berbagai model ResNet. Berikut contoh penggunaannya:

Listing 2.2. Menggunakan ResNet-18 untuk klasifikasi

```
import torchvision.models as models
import torch.nn as nn

model = models.resnet18(pretrained=True)

# Ubah layer output agar sesuai dengan CIFAR-10 (10 kelas)
num_ftrs = model.fc.in_features
model.fc = nn.Linear(num_ftrs, 10)
```

Jika ingin melakukan fine-tuning, kita bisa melatih ulang hanya layer akhir, atau seluruh model, tergantung pada kebutuhan dan ukuran dataset.

5. Arsitektur Lain yang Populer

Selain ResNet, berikut beberapa arsitektur CNN modern lainnya:

- **VGGNet**: arsitektur deep yang terdiri dari tumpukan conv layer 3x3. Akurasi tinggi tapi parameter banyak.
- **InceptionNet (GoogleNet)**: menggunakan *inception module* untuk menggabungkan beberapa ukuran kernel secara paralel.
- **MobileNet, EfficientNet**: arsitektur ringan dan efisien, cocok untuk deployment di perangkat mobile.

6. Perbandingan Arsitektur

Model	Jumlah Parameter	Akurasi ImageNet	Catatan
VGG-16	138M	71.5%	Banyak parameter
ResNet-50	25M	76.0%	Deep dengan residual
MobileNetV2	3.4M	72.0%	Ringan untuk mobile
EfficientNet-B0	5.3M	77.1%	Akurat dan ringan

Tabel 2.1. Perbandingan arsitektur CNN populer

Contoh Implementasi ResNet18 pada CIFAR-10

Berikut contoh penggunaan arsitektur `ResNet18` dari pustaka `torchvision` untuk klasifikasi gambar CIFAR-10:

Listing 2.3. Training ResNet18 untuk CIFAR-10

```
import torch
import torch.nn as nn
import torchvision
import torchvision.transforms as transforms
import torchvision.models as models
import torch.optim as optim

# 1. Transformasi dan DataLoader
transform = transforms.Compose([
    transforms.Resize(224),  # ResNet membutuhkan ukuran lebih besar
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
                                        download=True, transform=transform)
trainloader = torch.utils.data.DataLoader(trainset, batch_size=64,
                                          shuffle=True)

testset = torchvision.datasets.CIFAR10(root='./data', train=False,
                                       download=True, transform=transform)
testloader = torch.utils.data.DataLoader(testset, batch_size=100,
                                         shuffle=False)

# 2. Load pretrained ResNet18 dan sesuaikan output-nya
model = models.resnet18(pretrained=True)
model.fc = nn.Linear(model.fc.in_features, 10)  # CIFAR-10 = 10 kelas

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)

# 3. Optimizer dan Loss
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# 4. Training loop (5 epoch)
for epoch in range(5):
    model.train()
    running_loss = 0.0
    for inputs, labels in trainloader:
        inputs, labels = inputs.to(device), labels.to(device)

        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, labels)
```

```
            loss.backward()
            optimizer.step()

            running_loss += loss.item()
        print(f"Epoch {epoch+1}, Loss: {running_loss/len(trainloader):.4f}")

# 5. Evaluasi akurasi
model.eval()
correct = 0
total = 0
with torch.no_grad():
    for inputs, labels in testloader:
        inputs, labels = inputs.to(device), labels.to(device)
        outputs = model(inputs)
        _, predicted = torch.max(outputs, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

print(f"Akurasi data uji: {100 * correct / total:.2f}%")
```

—

Arsitektur modern seperti ResNet telah membuka jalan untuk pelatihan jaringan sangat dalam yang tetap stabil. Di bagian berikutnya, kita akan masuk ke topik yang lebih lanjut, yaitu **Object Detection** menggunakan model seperti YOLO.

BAB 3
Deteksi Objek

3.1 Pengantar Deteksi Objek

Deteksi objek (*object detection*) adalah tugas dalam computer vision untuk mengidentifikasi dan menentukan lokasi objek-objek tertentu dalam sebuah gambar atau video. Berbeda dengan klasifikasi citra yang hanya memberikan satu label untuk satu gambar, deteksi objek memberikan:

- **Label kelas objek**, misalnya: mobil, anjing, sepeda.
- **Lokasi objek**, biasanya dalam bentuk kotak pembatas (*bounding box*).

Contoh aplikasi deteksi objek:

- Sistem pengawasan dan keamanan (deteksi manusia/tindakan mencurigakan).
- Sistem bantuan mengemudi otomatis (mendeteksi kendaraan, rambu, pejalan kaki).
- Deteksi benda dalam citra medis (misalnya tumor).

3.2 Metrik Evaluasi untuk Object Detection

Evaluasi kinerja model object detection berbeda dengan klasifikasi biasa, karena model tidak hanya perlu memprediksi label objek, tetapi juga koordinat posisi kotak (bounding box) secara akurat. Oleh karena itu, dibutuhkan metrik khusus yang mempertimbangkan dua aspek tersebut.

3.2.1 Intersection over Union (IoU)

Intersection over Union (IoU) adalah metrik utama untuk mengevaluasi seberapa baik bounding box prediksi mencocokkan dengan ground truth.

$$\text{IoU} = \frac{\text{Luas dari irisan (intersection)}}{\text{Luas dari gabungan (union)}}$$

IoU memiliki nilai antara 0 dan 1. Semakin besar nilainya, semakin baik hasil prediksi.

Contoh:

Jika kotak prediksi dan ground truth saling tumpang tindih dengan area intersection sebesar 30 dan union sebesar 50, maka:

$$\text{IoU} = \frac{30}{50} = 0.6$$

Biasanya, prediksi dikatakan *benar* jika IoU-nya lebih besar dari threshold, misalnya 0.5.

3.2.2 Precision dan Recall

Setelah menggunakan threshold IoU (misalnya 0.5), kita bisa menghitung:

- **Precision** = Berapa banyak prediksi benar dari semua prediksi yang dibuat.
- **Recall** = Berapa banyak objek sebenarnya yang berhasil dideteksi.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Keterangan:

- TP: True Positive (prediksi benar, IoU cukup tinggi)
- FP: False Positive (prediksi salah atau IoU terlalu rendah)
- FN: False Negative (objek nyata tidak terdeteksi)

3.2.3 F1 Score

F1 Score adalah rata-rata harmonik dari precision dan recall. Berguna ketika ingin menyeimbangkan antara keduanya.

$$\text{F1} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

3.2.4 Average Precision (AP) dan mAP

Dalam praktik object detection, umumnya digunakan metrik Average Precision (AP), yaitu area di bawah kurva Precision–Recall. Kemudian dirata-ratakan untuk semua kelas (mAP — mean Average Precision).

Contoh: YOLOv5 dan model deteksi populer lainnya biasanya dilaporkan dengan mAP@IoU=0.5 atau mAP@[0.5:0.95] sebagai ukuran utama performa.

Referensi:

- https://learnopencv.com/evaluation-metrics-for-object-detection/
- https://towardsdatascience.com/intersection-over-union-iou-for-object-detection-61a85b0c1

Perbedaan dengan Klasifikasi Citra

Klasifikasi Citra	Deteksi Objek
Memberikan satu label untuk seluruh gambar	Memberikan label + posisi (bounding box) untuk setiap objek dalam gambar
Contoh output: `"Kucing"`	Contoh outtput: `["Kucing", (x1,y1,x2,y2)]`

Format Output: Bounding Box

Bounding box dinyatakan dengan koordinat (x_1, y_1) sebagai sudut kiri atas dan (x_2, y_2) sebagai sudut kanan bawah (atau bisa juga pusat (x, y), lebar dan tinggi (w, h) tergantung format).

3.3 YOLO: You Only Look Once

1. Konsep Dasar YOLO

YOLO (You Only Look Once) adalah pendekatan deteksi objek berbasis *single neural network* yang mengubah seluruh gambar input menjadi prediksi posisi dan kelas objek secara langsung. Berbeda dengan metode dua tahap seperti R-CNN (region proposal + klasifikasi), YOLO bersifat end-to-end dan sangat cepat.

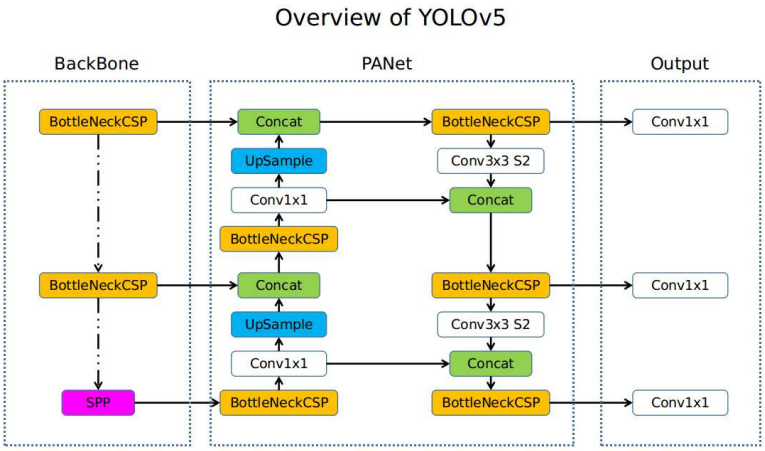
Prinsip utamanya:

- Gambar dibagi menjadi $S \times S$ grid.
- Setiap sel grid memprediksi:
  - B bounding box.
  - Confidence score.
  - Probabilitas kelas.
- Semua prediksi digabung dan di-*post-process* dengan teknik seperti Non-Maximum Suppression (NMS).

2. Arsitektur Model

YOLO menggunakan CNN sebagai backbone, misalnya:

- YOLOv1 – CNN custom
- YOLOv3/v4 – Darknet-53
- YOLOv5 – PyTorch + backbone CSPNet
- YOLOv8 – Model terbaru dari Ultralytics (2023)



Gambar 3.1. Arsitektur YOLOv5

Gambar 3.1 menunjukkan arsitektur YOLOv5.

3. Keunggulan YOLO

- Sangat cepat, cocok untuk aplikasi real-time.
- End-to-end training dan inference.
- Tidak memerlukan proposal region seperti R-CNN.

4. Implementasi YOLO dengan Ultralytics (YOLOv5/YOLOv8)

Salah satu implementasi YOLO yang paling mudah digunakan adalah dari `ultralytics`.

Langkah-langkah:

1. Install pustaka:

```
pip install ultralytics
```

2. Contoh penggunaan model pre-trained untuk inferensi:

```
from ultralytics import YOLO

model = YOLO("yolov8n.pt")  # gunakan model ringan

# Deteksi objek pada gambar
results = model("https://ultralytics.com/images/bus.jpg")

# Tampilkan hasil
results.show()
```

3. Output akan berupa gambar dengan bounding box dan label objek.

5. Dataset Deteksi Objek

YOLO sering digunakan pada dataset:

- **COCO**: Common Objects in Context – deteksi objek umum (manusia, hewan, kendaraan).
- **Pascal VOC**: dataset klasik dengan 20 kelas objek.
- **Custom dataset**: pengguna dapat melatih YOLO pada dataset sendiri dengan format yang sesuai.

6. Contoh Pelatihan Model YOLOv8 pada Dataset Sendiri (Opsional)

Ultralytics juga mendukung pelatihan pada dataset sendiri. Contohnya:

```
yolo task=detect mode=train model=yolov8n.pt data=data.yaml epochs=50 imgsz=640
```

File `data.yaml` harus berisi informasi: - lokasi data gambar dan label (format YOLO), - daftar kelas.

7. Fine-Tuning YOLOv8 pada Dataset Sendiri (Contoh Praktis)

Model YOLO dapat dilatih ulang (*fine-tuned*) pada dataset spesifik, misalnya untuk mendeteksi helm, wajah, kendaraan, dan lain-lain. Kita akan menggunakan dataset publik dari Roboflow yang sudah memiliki format label YOLO.

3.3.0.0.1 Contoh Dataset: Helmet Detection Dataset ini dapat diunduh dari:

- Roboflow: Helmet Detection Dataset
  https://universe.roboflow.com/jayz-workspace/helmet-detector-9rzmg

Klik **Download this Dataset** → pilih format `YOLOv8` dan salin perintah pip + 'yolo train'.

3.3.0.0.2 Langkah-langkah Fine-tuning:

1. **Install ultralytics** (jika belum):

   ```
   pip install ultralytics
   ```

2. **Unduh dataset dari Roboflow**:

   Setelah login Roboflow, klik download dataset → pilih format YOLOv8. Misalnya:

   ```
   # Script dari Roboflow
   curl -L "https://universe.roboflow.com/ds/abcd1234" > helmet.zip
   unzip helmet.zip -d helmet_dataset/
   ```

3. **Struktur folder dataset:**

   - `helmet_dataset/images/train/`

- `helmet_dataset/labels/train/`
- `helmet_dataset/data.yaml`

4. **Fine-tuning model pre-trained YOLOv8:**

```
yolo task=detect mode=train model=yolov8n.pt data=helmet_dataset/data.yaml epo
```

5. **Evaluasi dan inference:**

```
# Setelah pelatihan, gunakan model hasil fine-tune
yolo task=detect mode=predict model=runs/detect/train/weights/best.pt source=h
```

Model akan memproses gambar dan menyimpan hasil prediksi di folder `runs/detect/predict/`.

3.3.0.0.3 Tips:

- Gunakan `model=yolov8s.pt` atau `yolov8m.pt` untuk model yang lebih akurat.
- Untuk dataset kecil, gunakan augmentasi otomatis dari YOLO.
- Bisa juga menggunakan dataset lain dari Roboflow Universe.

—

8. Fine-tuning YOLOv8 Menggunakan Python (PyTorch API)

Selain menggunakan command line, pelatihan YOLOv8 juga dapat dilakukan secara langsung di Python menggunakan API dari pustaka `ultralytics`. Ini memudahkan integrasi dengan notebook seperti Jupyter atau Google Colab.

```
from ultralytics import YOLO

# Load model pre-trained (YOLOv8n paling ringan)
model = YOLO("yolov8n.pt")

# Lakukan pelatihan pada dataset lokal
model.train(data="helmet_dataset/data.yaml", epochs=20, imgsz=640)



# Evaluasi performa model
metrics = model.val()
print(metrics)



# Prediksi pada gambar baru
results = model("helmet_dataset/images/valid/sample1.jpg")
results[0].show()
```

```
# Simpan hasil prediksi
results[0].save(save_dir="predictions/")



# Akses model terbaik
best_model = YOLO("runs/detect/train/weights/best.pt")

# Gunakan untuk prediksi
best_model("helmet_dataset/images/test/img5.jpg").show()
```

3.3.0.0.9 Catatan:

- File `data.yaml` berisi informasi tentang path `train`, `val`, dan daftar kelas.
- Roboflow menyediakan dataset dengan format ini secara otomatis saat memilih ekspor YOLOv8.

9. Catatan Tambahan

Berikut beberapa contoh dataset lain dari Roboflow:

- Pothole Detection
  https://universe.roboflow.com/roboflow-100/pothole-detection-8yszz
- Dataset Umum Lainnya:
  https://universe.roboflow.com/object-detection

BAB 4

Segmentasi Citra (Image Segmentation)

Pendahuluan

Segmentasi citra adalah proses membagi suatu gambar digital menjadi beberapa segmen (kumpulan piksel) yang memiliki karakteristik serupa, seperti warna, intensitas, atau tekstur. Tujuannya adalah untuk menyederhanakan atau mengubah representasi citra menjadi bentuk yang lebih mudah dianalisis, misalnya untuk identifikasi objek, deteksi batas, atau pelacakan bagian tertentu dari gambar.

Berbeda dengan klasifikasi citra yang memberikan satu label untuk seluruh gambar, segmentasi memberikan label untuk setiap piksel, sehingga lebih presisi dan cocok untuk aplikasi seperti:

- Deteksi jalan dan marka pada sistem kendaraan otonom
- Segmentasi organ atau tumor pada citra medis
- Pemisahan latar dan objek pada aplikasi fotografi dan augmented reality

Secara umum, segmentasi dibagi menjadi dua jenis:

- **Semantic Segmentation:** setiap piksel diklasifikasikan ke dalam kelas tertentu (misalnya: jalan, mobil, pohon, langit).
- **Instance Segmentation:** mirip semantic segmentation, namun membedakan antara objek yang sama (misalnya 3 mobil = 3 instance).

—

4.1 Arsitektur Segmentasi Citra Berbasis Deep Learning

4.1.1 FCN (Fully Convolutional Network)

FCN adalah pengembangan dari CNN yang menggantikan layer fully-connected dengan convolutional sehingga outputnya berupa peta segmentasi (segmentation map), bukan nilai kelas tunggal.

- Encoder: bagian depan seperti CNN biasa untuk ekstraksi fitur.

- Decoder: upsampling bertahap untuk mengembalikan ukuran gambar asli.

4.1.2 U-Net

U-Net dirancang khusus untuk segmentasi biomedical. Arsitekturnya berbentuk "U" dengan:

- Downsampling (encoder): mirip CNN biasa.
- Upsampling (decoder): memperbesar resolusi.
- **Skip connections** menghubungkan layer encoder dan decoder yang sejajar, agar informasi spasial tetap terjaga.

4.1.3 SegNet

SegNet adalah arsitektur lain yang menggunakan **max-pooling indices** dari encoder untuk proses upsampling di decoder, mengurangi kompleksitas parameter.

4.1.4 DeepLab (v3, v3+)

DeepLab menggabungkan beberapa teknik canggih seperti:

- Atrous (dilated) convolutions untuk memperluas receptive field tanpa kehilangan resolusi.
- ASPP (Atrous Spatial Pyramid Pooling) untuk menangkap informasi multi-skala.
- Decoder module (pada v3+) untuk menghasilkan segmentasi tajam di tepi objek.

—

4.2 Implementasi Semantic Segmentation: Contoh dengan U-Net

Dataset Contoh: Oxford Pet Dataset

Dataset ini berisi gambar anjing dan kucing lengkap dengan mask segmentasi. Dataset tersedia di `torchvision.datasets`.

4.3 Implementasi U-Net dengan PyTorch

Arsitektur U-Net

U-Net merupakan salah satu arsitektur deep learning yang paling populer untuk tugas segmentasi citra. Desain arsitekturnya menyerupai huruf "U", dengan dua bagian utama:

- **Encoder (contracting path)**: Berfungsi mengekstraksi fitur melalui beberapa lapis convolution dan max-pooling.

- **Decoder (expanding path)**: Melakukan upsampling dengan transposed convolution, dan menggabungkan hasilnya dengan fitur dari encoder melalui skip connection.

Keunggulan U-Net adalah kemampuannya menggabungkan informasi spasial resolusi tinggi dari encoder dengan representasi semantik dari decoder, menghasilkan segmentasi yang akurat terutama pada batas objek.

Struktur Blok U-Net (PyTorch)

```
import torch
import torch.nn as nn

class UNet(nn.Module):
    def __init__(self, in_channels=3, out_channels=1):
        super(UNet, self).__init__()

        def conv_block(in_c, out_c):
            return nn.Sequential(
                nn.Conv2d(in_c, out_c, kernel_size=3, padding=1),
                nn.ReLU(inplace=True),
                nn.Conv2d(out_c, out_c, kernel_size=3, padding=1),
                nn.ReLU(inplace=True)
            )

        self.down1 = conv_block(in_channels, 64)
        self.pool1 = nn.MaxPool2d(2)

        self.down2 = conv_block(64, 128)
        self.pool2 = nn.MaxPool2d(2)

        self.bridge = conv_block(128, 256)

        self.up1 = nn.ConvTranspose2d(256, 128, kernel_size=2, stride=2)
        self.up_block1 = conv_block(256, 128)

        self.up2 = nn.ConvTranspose2d(128, 64, kernel_size=2, stride=2)
        self.up_block2 = conv_block(128, 64)

        self.final = nn.Conv2d(64, out_channels, kernel_size=1)

    def forward(self, x):
        d1 = self.down1(x)
        p1 = self.pool1(d1)

        d2 = self.down2(p1)
```

```
        p2 = self.pool2(d2)

        bridge = self.bridge(p2)

        up1 = self.up1(bridge)
        cat1 = torch.cat([up1, d2], dim=1)
        up1 = self.up_block1(cat1)

        up2 = self.up2(up1)
        cat2 = torch.cat([up2, d1], dim=1)
        up2 = self.up_block2(cat2)

        return self.final(up2)
```

Catatan Implementasi

- `torch.cat` digunakan untuk melakukan skip connection antara encoder dan decoder.
- Fungsi `nn.ConvTranspose2d` digunakan untuk melakukan upsampling dalam decoder.
- Output akhir menggunakan satu channel (`out_channels=1`) dengan aktivasi sigmoid (bisa ditambahkan jika diperlukan) untuk segmentasi biner.

Training Pipeline (Contoh Ringkas)

```
model = UNet(in_channels=3, out_channels=1).to(device)
criterion = nn.BCEWithLogitsLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)

for epoch in range(10):
    model.train()
    for images, masks in train_loader:
        images, masks = images.to(device), masks.to(device)
        preds = model(images)
        loss = criterion(preds, masks.float())
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    print(f"Epoch {epoch+1}, Loss: {loss.item():.4f}")
```

Evaluasi dan Visualisasi

Setelah pelatihan, prediksi pada gambar baru dapat divisualisasikan:

```
model.eval()
with torch.no_grad():
```

```
preds = torch.sigmoid(model(test_image.unsqueeze(0).to(device)))
pred_mask = preds.squeeze().cpu().numpy()
```

4.4 Implementasi U-Net dengan PyTorch

Arsitektur U-Net

U-Net merupakan salah satu arsitektur deep learning yang paling populer untuk tugas segmentasi citra. Desain arsitekturnya menyerupai huruf "U", dengan dua bagian utama:

- **Encoder (contracting path)**: Berfungsi mengekstraksi fitur melalui beberapa lapis convolution dan max-pooling.
- **Decoder (expanding path)**: Melakukan upsampling dengan transposed convolution, dan menggabungkan hasilnya dengan fitur dari encoder melalui skip connection.

Keunggulan U-Net adalah kemampuannya menggabungkan informasi spasial resolusi tinggi dari encoder dengan representasi semantik dari decoder, menghasilkan segmentasi yang akurat terutama pada batas objek.

Struktur Blok U-Net (PyTorch)

```
import torch
import torch.nn as nn

class UNet(nn.Module):
    def __init__(self, in_channels=3, out_channels=1):
        super(UNet, self).__init__()

        def conv_block(in_c, out_c):
            return nn.Sequential(
                nn.Conv2d(in_c, out_c, kernel_size=3, padding=1),
                nn.ReLU(inplace=True),
                nn.Conv2d(out_c, out_c, kernel_size=3, padding=1),
                nn.ReLU(inplace=True)
            )

        self.down1 = conv_block(in_channels, 64)
        self.pool1 = nn.MaxPool2d(2)

        self.down2 = conv_block(64, 128)
        self.pool2 = nn.MaxPool2d(2)

        self.bridge = conv_block(128, 256)
```

```
        self.up1 = nn.ConvTranspose2d(256, 128, kernel_size=2, stride=2)
        self.up_block1 = conv_block(256, 128)

        self.up2 = nn.ConvTranspose2d(128, 64, kernel_size=2, stride=2)
        self.up_block2 = conv_block(128, 64)

        self.final = nn.Conv2d(64, out_channels, kernel_size=1)

    def forward(self, x):
        d1 = self.down1(x)
        p1 = self.pool1(d1)

        d2 = self.down2(p1)
        p2 = self.pool2(d2)

        bridge = self.bridge(p2)

        up1 = self.up1(bridge)
        cat1 = torch.cat([up1, d2], dim=1)
        up1 = self.up_block1(cat1)

        up2 = self.up2(up1)
        cat2 = torch.cat([up2, d1], dim=1)
        up2 = self.up_block2(cat2)

        return self.final(up2)
```

Catatan Implementasi

- `torch.cat` digunakan untuk melakukan skip connection antara encoder dan decoder.
- Fungsi `nn.ConvTranspose2d` digunakan untuk melakukan upsampling dalam decoder.
- Output akhir menggunakan satu channel (`out_channels=1`) dengan aktivasi sigmoid (bisa ditambahkan jika diperlukan) untuk segmentasi biner.

Training Pipeline (Contoh Ringkas)

```
model = UNet(in_channels=3, out_channels=1).to(device)
criterion = nn.BCEWithLogitsLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)

for epoch in range(10):
    model.train()
    for images, masks in train_loader:
        images, masks = images.to(device), masks.to(device)
```

```
        preds = model(images)
        loss = criterion(preds, masks.float())
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    print(f"Epoch {epoch+1}, Loss: {loss.item():.4f}")
```

Evaluasi dan Visualisasi

Setelah pelatihan, prediksi pada gambar baru dapat divisualisasikan:

```
model.eval()
with torch.no_grad():
    preds = torch.sigmoid(model(test_image.unsqueeze(0).to(device)))
    pred_mask = preds.squeeze().cpu().numpy()
```

4.5 Instance Segmentation dan Mask R-CNN

Konsep Instance Segmentation

Instance segmentation adalah penggabungan dari object detection dan semantic segmentation. Tujuan utamanya adalah:

- Mendeteksi setiap objek (instance) pada gambar beserta kelasnya.
- Memberikan mask segmentasi yang berbeda untuk setiap instance, meskipun berada dalam kelas yang sama.

Contoh:

- Pada semantic segmentation, semua piksel kucing akan diberi label "kucing".
- Pada instance segmentation, setiap kucing (jika ada lebih dari satu) akan diberi mask yang terpisah.

Mask R-CNN

Mask R-CNN merupakan model instance segmentation yang populer, dikembangkan oleh Facebook AI Research (FAIR). Mask R-CNN memperluas Faster R-CNN dengan:

- Menambahkan cabang baru untuk prediksi **mask biner** untuk setiap Region of Interest (RoI).
- Memiliki output: *bounding box*, kelas objek, dan mask segmentasi.

Arsitektur umumnya terdiri dari:

- Backbone CNN (ResNet + FPN) untuk ekstraksi fitur.

- Region Proposal Network (RPN) untuk menghasilkan kandidat objek.
- RoIAlign untuk ekstraksi fitur terlokalisasi.
- Head untuk prediksi kelas, bbox, dan mask.

Implementasi Mask R-CNN Pretrained (PyTorch)

`torchvision` menyediakan model pretrained Mask R-CNN yang bisa langsung digunakan.

```
import torchvision
from torchvision.models.detection import maskrcnn_resnet50_fpn
from torchvision.transforms import functional as F
from PIL import Image
import matplotlib.pyplot as plt
import torch

# Load model pretrained
model = maskrcnn_resnet50_fpn(pretrained=True)
model.eval()

# Load dan transform gambar
img = Image.open("sample.jpg").convert("RGB")
img_tensor = F.to_tensor(img).unsqueeze(0)

# Prediksi
with torch.no_grad():
    pred = model(img_tensor)[0]

# Visualisasi prediksi
for i in range(len(pred["boxes"])):
    score = pred["scores"][i].item()
    if score > 0.8:
        bbox = pred["boxes"][i]
        mask = pred["masks"][i, 0].cpu().numpy()
        label = pred["labels"][i].item()
        print(f"Objek: {label}, Confidence: {score:.2f}")
```

Catatan:

- Label dapat dipetakan ke nama kelas COCO dengan `coco.names`.
- Mask R-CNN membutuhkan cukup banyak memori, sebaiknya jalankan di GPU.

BAB 5

Topik Lanjut dalam Computer Vision

5.1 Transfer Learning dan Model Pretrained

Transfer learning adalah teknik penting dalam deep learning di mana sebuah model yang telah dilatih sebelumnya (pretrained model) pada dataset besar seperti ImageNet digunakan kembali untuk tugas yang berbeda. Ide dasarnya adalah bahwa model telah mempelajari representasi umum (seperti deteksi tepi, bentuk dasar, dan pola) yang juga bermanfaat untuk tugas baru dengan dataset yang lebih kecil.

Pendekatan ini sangat membantu ketika:

- Dataset baru terlalu kecil untuk melatih model dari nol.
- Ingin memanfaatkan kekuatan model besar seperti ResNet, EfficientNet, VGG, dan lain-lain.

Ada dua pendekatan umum:

1. **Feature Extraction**: Hanya menggunakan bagian awal dari model pretrained sebagai ekstraktor fitur, lalu melatih classifier baru di atasnya.
2. **Fine-tuning**: Melatih ulang beberapa atau seluruh layer dari model pretrained agar menyesuaikan dengan data baru.

5.2 Fine-Tuning Model untuk Dataset Baru

Fine-tuning dilakukan dengan memuat model pretrained, kemudian menyesuaikan struktur akhirnya (biasanya layer klasifikasi) dan mengatur parameter yang ingin dilatih ulang.

Contoh kode berikut menunjukkan cara fine-tuning model ResNet pada dataset baru:

```
import torchvision.models as models
import torch.nn as nn

model = models.resnet18(pretrained=True)
for param in model.parameters():
    param.requires_grad = False  # freeze semua layer
```

```
# Ganti classifier terakhir
num_ftrs = model.fc.in_features
model.fc = nn.Linear(num_ftrs, 10)  # misalnya untuk 10 kelas baru

# Sekarang hanya model.fc yang akan dilatih
optimizer = torch.optim.Adam(model.fc.parameters(), lr=1e-3)
```

Kita juga bisa memilih untuk melatih kembali sebagian layer awal (unfreeze bertahap) jika dataset cukup besar dan memiliki domain yang berbeda dari ImageNet.

5.3 Augmentasi Data

Augmentasi data (data augmentation) adalah proses memodifikasi data pelatihan untuk meningkatkan keragaman dan membantu model generalisasi lebih baik. Teknik ini sangat penting terutama ketika dataset terbatas.

Beberapa teknik umum:

- Rotasi acak
- Flip horizontal/vertikal
- Pemotongan dan pergeseran
- Perubahan brightness, kontras, noise

Implementasi di PyTorch:

```
from torchvision import transforms

transform = transforms.Compose([
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(10),
    transforms.ColorJitter(brightness=0.2, contrast=0.2),
    transforms.ToTensor()
])
```

Dengan augmentasi ini, model menjadi lebih robust terhadap variasi posisi, pencahayaan, dan orientasi objek dalam gambar.

5.4 Visualisasi dan Interpretasi CNN

Meskipun CNN bekerja sangat baik dalam klasifikasi dan deteksi objek, interpretasi model ini tetap menjadi tantangan. Visualisasi CNN membantu kita memahami apa yang dipelajari model dan bagaimana ia membuat keputusan.

Beberapa teknik populer:

- **Visualisasi filter convolution**: menunjukkan pola seperti tepi atau tekstur yang ditangkap oleh setiap filter.

- **Feature map**: hasil aktivasi pada layer tertentu, menunjukkan area penting dari input.
- **Grad-CAM (Gradient-weighted Class Activation Mapping)**: menunjukkan area gambar mana yang paling berkontribusi pada prediksi model.

Grad-CAM dapat digunakan untuk menjelaskan keputusan model CNN dalam klasifikasi gambar.

Contoh visualisasi interaktif CNN dapat dilihat di situs:

- https://poloclub.github.io/cnn-explainer/

5.5 Arsitektur Modern dan Perkembangan Terbaru

Seiring berkembangnya teknologi, arsitektur CNN juga terus berevolusi. Beberapa arsitektur modern yang populer meliputi:

- **ResNet**: menggunakan koneksi residual (shortcut) untuk mengatasi masalah vanishing gradient.
- **EfficientNet**: menggabungkan teknik scaling secara efisien untuk mencapai akurasi tinggi dengan parameter lebih sedikit.
- **Vision Transformer (ViT)**: menerapkan prinsip attention (yang awalnya populer di NLP) untuk gambar. Meskipun tidak menggunakan convolutions, ViT mampu menyaingi CNN dalam berbagai tugas klasifikasi.

Catatan: Transformer akan dibahas lebih lanjut dalam modul NLP.

5.6 Aplikasi Computer Vision di Dunia Nyata

Computer Vision telah diimplementasikan secara luas dalam berbagai bidang, antara lain:

- **Kendaraan otonom (self-driving cars)**: mendeteksi kendaraan lain, pejalan kaki, rambu lalu lintas, dan marka jalan.
- **Sistem pengenalan wajah (face recognition)**: untuk keamanan, login otomatis, dan pelacakan.
- **Medis**: analisis citra X-ray, CT-scan, dan MRI untuk deteksi penyakit.
- **Industri dan manufaktur**: inspeksi kualitas produk secara otomatis.
- **Smart farming dan lingkungan**: monitoring pertanian, deteksi gulma, pemantauan hutan dan satwa liar.

Dengan meningkatnya ketersediaan data dan kekuatan komputasi, penerapan Computer Vision akan semakin luas dan berdampak besar dalam berbagai aspek kehidupan.

5.7 Referensi dan Bahan Bacaan Lanjutan

Untuk memperdalam pemahaman tentang topik lanjutan dalam computer vision, berikut adalah beberapa sumber bacaan dan tutorial yang direkomendasikan:

- **Transfer Learning and Fine-Tuning**
  - https://pytorch.org/tutorials/beginner/transfer_learning_tutorial.html
  - https://www.tensorflow.org/tutorials/images/transfer_learning
- **Visualisasi CNN dan Grad-CAM**
  - Interaktif: https://poloclub.github.io/cnn-explainer/
  - Penjelasan Grad-CAM: https://keras.io/examples/vision/grad_cam/
- **Arsitektur CNN Modern**
  - ResNet: https://arxiv.org/abs/1512.03385
  - EfficientNet: https://arxiv.org/abs/1905.11946
- **Vision Transformer**
  - Ringkasan konsep ViT: https://theaisummer.com/vision-transformer/
- **Aplikasi CV di Dunia Nyata**
  - Buku: *Deep Learning for Computer Vision* by Rajalingappaa Shanmugamani
  - Artikel dan studi kasus industri: https://paperswithcode.com/area/computer-vision

Bibliografi

[DBCR19] David M Diez, Christopher D Barr, and Mine Cetinkaya-Rundel, *Openintro statistics*, 3rd ed., OpenIntro, 2019.

[Dev23] Google Developers, *Machine learning crash course*, https://developers.google.com/machine-learning/crash-course, 2023.

[DFO20] Marc Peter Deisenroth, A Aldo Faisal, and Cheng Soon Ong, *Mathematics for machine learning*, Cambridge University Press, 2020.

[GBC16] Ian Goodfellow, Yoshua Bengio, and Aaron Courville, *Deep learning*, MIT Press, 2016, Book in preparation for MIT Press.

[HTF01] Trevor Hastie, Robert Tibshirani, and Jerome Friedman, *The elements of statistical learning*, Springer Series in Statistics, Springer New York Inc., New York, NY, USA, 2001.

[ID17] Barbara Illowsky and Susan Dean, *Introductory statistics*, OpenStax, 2017.

[JWHT13] Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani, *An introduction to statistical learning*, Springer, 2013.

[Uni23] Duke University, *Data science math skills*, https://www.coursera.org/learn/datasciencemathskills, 2023.

[Was04] Larry Wasserman, *All of statistics: A concise course in statistical inference*, Springer, 2004.